

RL_02_dqn

April 10, 2026

1 Reinforcement Learning - Parte 2: Deep RL

Prerequisites: Haber completado `RL_01_fundamentos.ipynb` (Parte 1)

1.1 Serie Completa

Notebook	Contenido	Estado
<code>RL_01_fundamentos.ipynb</code>	Fundamentos, SARSA, Q-Learning	Completado
<code>RL_02_dqn.ipynb</code> (este)	DQN, Gymnasium, Práctica	Estás aquí
<code>RL_03_sb3.ipynb</code>	Stable-Baselines3, PPO, SAC	Siguiente

1.2 Índice de este Notebook

8. Deep Q-Network (DQN) - Q-Learning + Redes Neuronales
9. Gymnasium: Entornos de RL - Entornos estándar
10. Práctica Final - Implementación completa

1.3 ¿Por qué separar?

La **Parte 1** cubre RL tabular donde: - Estados son discretos y finitos - $Q(s,a)$ se almacena en una tabla/diccionario - No se necesitan redes neuronales

La **Parte 2** (este notebook) introduce Deep RL donde: - Estados pueden ser continuos o de alta dimensionalidad - $Q(s,a)$ se aproxima con una red neuronal - Se requiere PyTorch

```
[46]: # Imports necesarios para Deep RL
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import defaultdict, deque

# Deep Learning
try:
    import torch
    import torch.nn as nn
```

```

import torch.optim as optim
TORCH_AVAILABLE = True
print(f'PyTorch {torch.__version__} disponible')
except ImportError:
    TORCH_AVAILABLE = False
    print('PyTorch no disponible. Instalar con: pip install torch')

# Gymnasium
try:
    import gymnasium as gym
    print(f'Gymnasium {gym.__version__} disponible')
except ImportError:
    print('Gymnasium no disponible. Instalar con: pip install gymnasium')

# Configuración de visualización
plt.rcParams['figure.figsize'] = [10, 6]
plt.rcParams['figure.dpi'] = 100

```

PyTorch 2.11.0+cu130 disponible

Gymnasium 1.2.3 disponible

```

[47]: # EJEMPLO PRÁCTICO 9: DQN Simple
# =====
# Deep Q-Network: Q-Learning + Redes Neuronales
# Permite manejar estados continuos o de alta dimensionalidad.

# Verificar si PyTorch está disponible
try:
    import torch                # Framework de deep learning
    import torch.nn as nn       # Módulo de redes neuronales
    import torch.optim as optim # Optimizadores (Adam, SGD, etc.)
    TORCH_AVAILABLE = True
    print(" PyTorch disponible")
except ImportError:
    TORCH_AVAILABLE = False
    print(" PyTorch no está instalado. Ejecuta: pip install torch")
    print(" Las celdas de DQN no funcionarán sin PyTorch.")

from collections import deque # Cola de doble extremo (eficiente para buffers)

if TORCH_AVAILABLE:

    class ReplayBuffer:
        """
        Buffer de experiencias para Experience Replay.

        ¿Por qué Experience Replay?

```

```

1. ROMPE CORRELACIÓN: Las experiencias consecutivas están
↪ correlacionadas
    (estado t+1 depende de t). Muestrear al azar rompe esta correlación.
2. REUTILIZA DATOS: Cada experiencia se puede usar múltiples veces.
3. ESTABILIDAD: El entrenamiento es más estable con batches diversos.
"""

def __init__(self, capacidad=10000):
    """
    Args:
        capacidad: Máximo de experiencias a almacenar.
                  Cuando se llena, las más antiguas se descartan.
    """
    # deque con maxlen descarta automáticamente elementos antiguos
    self.buffer = deque(maxlen=capacidad)

def agregar(self, estado, accion, recompensa, siguiente_estado,
↪ terminado):
    """
    Guarda una transición (s, a, r, s', done) en el buffer.

    Args:
        estado: Estado donde se tomó la acción
        accion: Acción ejecutada
        recompensa: Recompensa recibida
        siguiente_estado: Estado resultante
        terminado: Si el episodio terminó
    """
    # Guardar la tupla completa
    self.buffer.append((estado, accion, recompensa, siguiente_estado,
↪ terminado))

def sample(self, batch_size):
    """
    Muestra un batch aleatorio de experiencias.

    Args:
        batch_size: Número de experiencias a muestrear

    Returns:
        Tupla de arrays: (estados, acciones, recompensas, siguientes,
↪ terminados)
    """
    # random.sample elige sin repetición
    batch = random.sample(self.buffer, batch_size)

    # Desempaquetar: convertir lista de tuplas a tuplas de listas

```

```

    estados, acciones, recompensas, siguientes, terminados = zip(*batch)

    # Convertir a arrays numpy para eficiencia
    return (np.array(estados), np.array(acciones), np.
↪ array(recompensas),
        np.array(siguientes), np.array(terminados))

def __len__(self):
    """Devuelve el número de experiencias almacenadas."""
    return len(self.buffer)

class DQN(nn.Module):
    """
    Red neuronal que aproxima la función  $Q(s, a)$ .

    Arquitectura:
    Input (estado) → Capa 1 (64) → ReLU → Capa 2 (64) → ReLU → Output ↪
    ↪ (Q-valores)

    En lugar de una tabla  $Q[s][a]$ , usamos una red que:
    - Recibe: vector de estado (ej: [posición_x, posición_y])
    - Devuelve: Q-valor para cada acción [ $Q(s, a_0)$ ,  $Q(s, a_1)$ ,  $Q(s, a_2)$ , ↪
    ↪  $Q(s, a_3)$ ]
    """

    def __init__(self, input_size, n_acciones):
        """
        Args:
            input_size: Dimensión del vector de estado
            n_acciones: Número de acciones posibles
        """
        super(DQN, self).__init__() # Inicializar clase padre

        # Definir la arquitectura como secuencia de capas
        self.red = nn.Sequential(
            # Capa 1: entrada → 64 neuronas
            nn.Linear(input_size, 64),
            nn.ReLU(), # Activación no lineal

            # Capa 2: 64 → 64 neuronas
            nn.Linear(64, 64),
            nn.ReLU(),

            # Capa de salida: 64 → n_acciones
            # Cada salida es el Q-valor de una acción
            nn.Linear(64, n_acciones)

```

```

        # Sin activación: Q-valores pueden ser negativos
    )

def forward(self, x):
    """
    Propagación hacia adelante.

    Args:
        x: Tensor de estados (batch_size, input_size)

    Returns:
        Q-valores para cada acción (batch_size, n_acciones)
    """
    return self.red(x)

class AgenteDQN:
    """
    Agente DQN completo con Experience Replay y Target Network.

    Componentes clave:
    1. Q-Network: Red que aprende (se actualiza cada paso)
    2. Target Network: Copia de Q que se actualiza periódicamente
    3. Replay Buffer: Almacena experiencias para muestreo aleatorio
    """

    def __init__(self, input_size, n_acciones, lr=0.001, gamma=0.99,
        ↪epsilon=1.0):
        """
        Args:
            input_size: Dimensión del estado
            n_acciones: Número de acciones posibles
            lr: Learning rate para el optimizador
            gamma: Factor de descuento
            epsilon: Probabilidad inicial de exploración
        """
        self.n_acciones = n_acciones
        self.gamma = gamma          # Factor de descuento
        self.epsilon = epsilon      # Exploración inicial (100%)
        self.epsilon_min = 0.01    # Exploración mínima (1%)
        self.epsilon_decay = 0.995 # Factor de decaimiento

        # RED Q (la que aprende)
        self.q_network = DQN(input_size, n_acciones)

        # RED TARGET (para estabilidad)
        # Se inicializa igual a Q y se actualiza periódicamente

```

```

self.target_network = DQN(input_size, n_acciones)
self.target_network.load_state_dict(self.q_network.state_dict())

# OPTIMIZADOR: Adam es estándar para deep learning
self.optimizer = optim.Adam(self.q_network.parameters(), lr=lr)

# FUNCIÓN DE PÉRDIDA: MSE entre Q predicho y target
self.loss_fn = nn.MSELoss()

# REPLAY BUFFER
self.memory = ReplayBuffer()

def estado_a_tensor(self, estado):
    """
    Convierte el estado (tupla) a tensor de PyTorch.

    Args:
        estado: Tupla (fila, columna)

    Returns:
        Tensor normalizado [0, 1]
    """
    # Normalizar coordenadas a rango [0, 1]
    # Esto ayuda a la red a aprender más rápido
    return torch.FloatTensor([estado[0]/3, estado[1]/3])

def seleccionar_accion(self, estado):
    """
    Selecciona acción usando política -greedy.

    Args:
        estado: Estado actual

    Returns:
        Índice de la acción seleccionada
    """
    # Explorar con probabilidad epsilon
    if random.random() < self.epsilon:
        return random.randint(0, self.n_acciones - 1)

    # Explotar: usar la red para elegir la mejor acción
    with torch.no_grad(): # No calcular gradientes (solo inferencia)
        estado_tensor = self.estado_a_tensor(estado).unsqueeze(0)
        q_valores = self.q_network(estado_tensor)
        return q_valores.argmax().item() # Índice del máximo

```

```

def recordar(self, estado, accion, recompensa, siguiente_estado,
↪terminado):
    """Guarda experiencia en el replay buffer."""
    self.memory.agregar(estado, accion, recompensa, siguiente_estado,
↪terminado)

def entrenar(self, batch_size=32):
    """
    Entrena la red Q con un batch del replay buffer.

    Este es el núcleo del algoritmo DQN.

    Args:
        batch_size: Número de experiencias para entrenar

    Returns:
        loss: Pérdida del entrenamiento (para monitoreo)
    """

    # No entrenar si no hay suficientes experiencias
    if len(self.memory) < batch_size:
        return 0

    # PASO 1: Muestrear batch aleatorio del buffer
    estados, acciones, recompensas, siguientes, terminados = self.
↪memory.sample(batch_size)

    # PASO 2: Convertir a tensores de PyTorch
    estados_t = torch.FloatTensor([[s[0]/3, s[1]/3] for s in estados])
    acciones_t = torch.LongTensor(acciones)
    recompensas_t = torch.FloatTensor(recompensas)
    siguientes_t = torch.FloatTensor([[s[0]/3, s[1]/3] for s in
↪siguientes])
    terminados_t = torch.FloatTensor(terminados)

    # PASO 3: Calcular Q-valores actuales para las acciones tomadas
    # q_network(estados) devuelve Q para todas las acciones
    # gather selecciona solo las Q de las acciones que se tomaron
    q_actuales = self.q_network(estados_t).gather(1, acciones_t.
↪unsqueeze(1))

    # PASO 4: Calcular targets usando la TARGET network
    # Esta es la clave de DQN: usar red separada para estabilidad
    with torch.no_grad():
        # Máximo Q del siguiente estado (según target network)
        q_siguientes = self.target_network(siguientes_t).max(1)[0]
        # Si terminó, no hay futuro
        q_siguientes = q_siguientes * (1 - terminados_t)

```

```

        # Target = r +  $\gamma$  * max Q(s', a')
        targets = recompensas_t + self.gamma * q_siguietes

    # PASO 5: Calcular pérdida (diferencia entre predicción y target)
    loss = self.loss_fn(q_actuales.squeeze(), targets)

    # PASO 6: Backpropagation
    self.optimizer.zero_grad() # Limpiar gradientes anteriores
    loss.backward()             # Calcular gradientes
    self.optimizer.step()        # Actualizar pesos

    return loss.item()

def actualizar_target(self):
    """
    Copia los pesos de q_network a target_network.

    Se hace periódicamente (cada N episodios) para estabilidad.
    """
    self.target_network.load_state_dict(self.q_network.state_dict())

def decay_epsilon(self):
    """Reduce epsilon gradualmente (explorar menos con el tiempo)."""
    self.epsilon = max(self.epsilon_min, self.epsilon * self.
↪epsilon_decay)

print("\n Clases DQN definidas correctamente")
print("  - ReplayBuffer: Almacena experiencias")
print("  - DQN: Red neuronal para Q-valores")
print("  - AgenteDQN: Agente completo con entrenamiento")

```

PyTorch disponible

Clases DQN definidas correctamente

- ReplayBuffer: Almacena experiencias
- DQN: Red neuronal para Q-valores
- AgenteDQN: Agente completo con entrenamiento

```

[48]: class GridWorld:
    """
    Un mundo de cuadrícula simple donde el agente debe llegar a la meta.

    Mapa 4x4:
    +---+---+---+---+
    | S | . | . | . |   S = Start (inicio, posición 0,0)
    +---+---+---+---+
    """

```

```

| · | X | · | · |    X = Obstáculo (pared, no se puede atravesar)
+---+---+---+---+
| · | · | · | · |    · = Celda vacía (se puede caminar)
+---+---+---+---+
| · | · | · | G |    G = Goal (meta, posición 3,3)
+---+---+---+---+

```

Recompensas:

- Llegar a G: +10 (y termina el episodio)
 - Cada paso: -1 (incentiva encontrar el camino más corto)
- """

```

def __init__(self):
    """Inicializa el entorno con sus parámetros fijos."""
    # Tamaño de la cuadrícula (4x4 = 16 estados posibles)
    self.grid_size = 4

    # Posiciones especiales (fila, columna)
    self.start = (0, 0)      # Esquina superior izquierda
    self.goal = (3, 3)      # Esquina inferior derecha
    self.obstacle = (1, 1)  # Un obstáculo en el camino

    # Estado actual del agente (empieza en start)
    self.state = self.start

    # DEFINICIÓN DE ACCIONES
    # Cada acción es un desplazamiento (delta_fila, delta_columna)
    self.actions = {
        0: (-1, 0), # Arriba: fila -1, columna igual
        1: (0, 1),  # Derecha: fila igual, columna +1
        2: (1, 0),  # Abajo: fila +1, columna igual
        3: (0, -1)  # Izquierda: fila igual, columna -1
    }

    # Nombres para visualización
    self.action_names = ['↑', '→', '↓', '←']

def reset(self):
    """
    Reinicia el entorno al estado inicial.

    Se llama al inicio de cada episodio.

    Returns:
        state: El estado inicial (posición de inicio)
    """
    self.state = self.start # Volver a la posición inicial

```

```

return self.state          # El agente necesita saber dónde está

def step(self, action):
    """
    Ejecuta una acción y devuelve el resultado.

    Esta es la función principal del entorno. Implementa la
    dinámica del mundo: cómo responde a las acciones del agente.

    Args:
        action: Índice de la acción (0=↑, 1=→, 2=↓, 3=←)

    Returns:
        new_state: Nueva posición tras la acción
        reward: Recompensa recibida
        done: True si el episodio terminó (llegó a meta)
    """
    # PASO 1: Calcular la nueva posición tentativa
    delta = self.actions[action] # Obtener desplazamiento
    new_row = self.state[0] + delta[0] # Nueva fila
    new_col = self.state[1] + delta[1] # Nueva columna
    new_state = (new_row, new_col)

    # PASO 2: Verificar si el movimiento es válido
    # Un movimiento es válido si:
    # - No sale de los límites de la cuadrícula
    # - No choca contra el obstáculo
    dentro_lmites = (0 <= new_row < self.grid_size and
                     0 <= new_col < self.grid_size)
    no_es_obstaculo = (new_state != self.obstacle)

    if dentro_lmites and no_es_obstaculo:
        # Movimiento válido: actualizar posición
        self.state = new_state
    # Si no es válido, el agente se queda donde estaba

    # PASO 3: Calcular recompensa según el estado resultante
    if self.state == self.goal:
        reward = 10 # ¡Llegó a la meta! Recompensa alta
        done = True # El episodio termina
    else:
        reward = -1 # Penalización por paso (incentiva rapidez)
        done = False # El episodio continúa

    return self.state, reward, done

def render(self):

```

```

"""
Visualiza el estado actual del entorno en la consola.

Útil para debugging y para ver qué está haciendo el agente.
"""

# Crear grid vacío
grid = [['.' for _ in range(self.grid_size)]
        for _ in range(self.grid_size)]

# Marcar posiciones especiales
grid[self.obstacle[0]][self.obstacle[1]] = 'X' # Obstáculo
grid[self.goal[0]][self.goal[1]] = 'G'         # Meta
grid[self.state[0]][self.state[1]] = 'A'       # Agente

# Imprimir
print("\n" + "-" * (self.grid_size * 2 + 1))
for row in grid:
    print("|" + " ".join(row) + "|")
print("-" * (self.grid_size * 2 + 1))

```

```

[49]: def entrenar_dqn(env, agente, n_episodios=500, max_pasos=1000, verbose=True):
    """
    Entrena un agente DQN.

    Args:
        env: Entorno de Gymnasium
        agente: Instancia de DQNAgent
        n_episodios: Número de episodios
        max_pasos: Máximo de pasos por episodio
        verbose: Si mostrar progreso

    Returns:
        Tupla (recompensas, perdidas) con historiales
    """
    recompensas = []
    perdidas = []

    for ep in range(n_episodios):
        estado, _ = env.reset()
        recompensa_total = 0
        perdida_total = 0
        n_updates = 0

        for paso in range(max_pasos):
            accion = agente.seleccionar_accion(estado)
            siguiente, recompensa, terminado, truncado, _ = env.step(accion)
            done = terminado or truncado

```

```

    agente.recordar(estado, accion, recompensa, siguiente, done)
    loss = agente.entrenar_paso()

    recompensa_total += recompensa
    if loss > 0:
        perdida_total += loss
        n_updates += 1

    estado = siguiente

    if done:
        break

    agente.fin_episodio()
    recompensas.append(recompensa_total)
    perdidas.append(perdida_total / max(n_updates, 1))

    if verbose and (ep + 1) % 50 == 0:
        promedio = np.mean(recompensas[-50:])
        print(f"Episodio {ep + 1:4d} | Recompensa: {recompensa_total:7.1f}┐
↪ | "
           f"Promedio: {promedio:7.2f} | : {agente.epsilon:.3f}")

    return recompensas, perdidas

```

[50]: # EJEMPLO PRÁCTICO 10: Entrenar DQN en GridWorld

```

if not TORCH_AVAILABLE:
    print(" PyTorch no disponible. Salta esta celda.")
else:
    def entrenar_dqn(env, n_episodios=300):
        """Entrena un agente DQN"""
        agente = AgenteDQN(input_size=2, n_acciones=4)

        recompensas_historial = []
        perdidas_historial = []

        for episodio in range(n_episodios):
            estado = env.reset()
            recompensa_total = 0
            perdida_total = 0
            pasos = 0

            for _ in range(100):
                accion = agente.seleccionar_accion(estado)
                siguiente, recompensa, terminado = env.step(accion)

```

```

        agente.recordar(estado, accion, recompensa, siguiente,
↪terminado)

        perdida = agente.entrenar(batch_size=32)

        recompensa_total += recompensa
        perdida_total += perdida
        pasos += 1
        estado = siguiente

        if terminado:
            break

        # Actualizar target network cada 10 episodios
        if episodio % 10 == 0:
            agente.actualizar_target()

        agente.decay_epsilon()

        recompensas_historial.append(recompensa_total)
        perdidas_historial.append(perdida_total / max(pasos, 1))

        if episodio % 50 == 0:
            promedio = np.mean(recompensas_historial[-50:]) if
↪len(recompensas_historial) >= 50 else np.mean(recompensas_historial)
            print(f"Episodio {episodio:3d} | Recompensa: {recompensa_total:
↪6.1f} | "
                  f"Promedio: {promedio:6.2f} | : {agente.epsilon:.3f}")

        return agente, recompensas_historial, perdidas_historial

# Entrenar
print("=" * 60)
print("ENTRENAMIENTO DQN")
print("=" * 60)

env = GridWorld()
agente_dqn, recompensas_dqn, perdidas_dqn = entrenar_dqn(env,
↪n_episodios=300)

# Visualizar
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

window = 20
recompensas_suav = np.convolve(recompensas_dqn, np.ones(window)/window,
↪mode='valid')

```

```

axes[0].plot(recompensas_suav)
axes[0].set_xlabel('Episodio')
axes[0].set_ylabel('Recompensa (media móvil)')
axes[0].set_title('Curva de Aprendizaje DQN')

axes[1].plot(perdidas_dqn, color='red', alpha=0.5)
axes[1].set_xlabel('Episodio')
axes[1].set_ylabel('Pérdida (Loss)')
axes[1].set_title('Pérdida durante Entrenamiento')

plt.tight_layout()
plt.show()

print(f"\n DQN entrenado exitosamente")
print(f"Recompensa promedio final: {np.mean(recompensas_dqn[-50:]):.2f}")

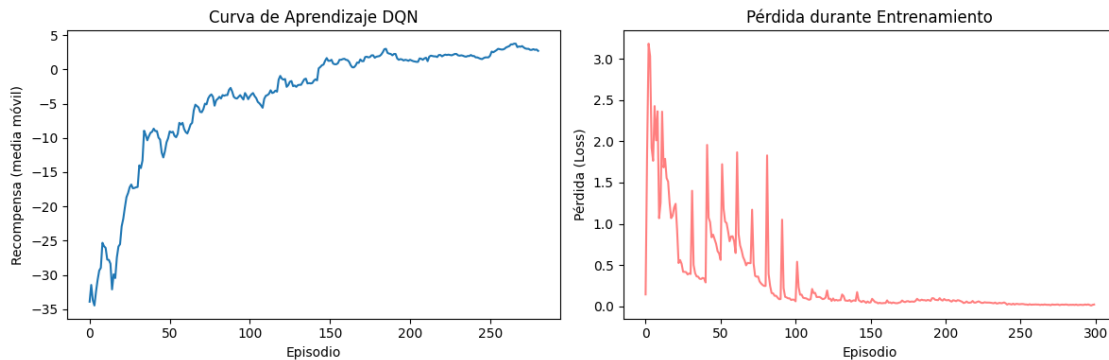
```

=====

ENTRENAMIENTO DQN

=====

Episodio	0		Recompensa:	-69.0		Promedio:	-69.00		:	0.995
Episodio	50		Recompensa:	-4.0		Promedio:	-23.14		:	0.774
Episodio	100		Recompensa:	-3.0		Promedio:	-7.10		:	0.603
Episodio	150		Recompensa:	1.0		Promedio:	-2.98		:	0.469
Episodio	200		Recompensa:	4.0		Promedio:	1.66		:	0.365
Episodio	250		Recompensa:	-7.0		Promedio:	1.56		:	0.284



DQN entrenado exitosamente

Recompensa promedio final: 2.68

```

[51]: # EJEMPLO PRÁCTICO 11: Explorando Gymnasium

# Instalar gymnasium si no está disponible

```

```

try:
    import gymnasium as gym
except ModuleNotFoundError:
    print("Instalando gymnasium...")
    !pip install gymnasium -q
    import gymnasium as gym

print("=" * 60)
print("EXPLORANDO GYMNASIUM")
print("=" * 60)

# Crear entorno CartPole
env = gym.make("CartPole-v1")

print("\n Entorno: CartPole-v1")
print("-" * 40)
print(f"Espacio de observación: {env.observation_space}")
print(f" - Tipo: {type(env.observation_space).__name__}")
print(f" - Forma: {env.observation_space.shape}")
print(f" - Límites: {env.observation_space.low} a {env.observation_space.
↪high}")

print(f"\nEspacio de acciones: {env.action_space}")
print(f" - Número de acciones: {env.action_space.n}")
print(f" - Acciones: 0 = empujar izquierda, 1 = empujar derecha")

# Reset
observacion, info = env.reset(seed=42)
print(f"\n Después de reset():")
print(f" Observación: {observacion}")
print(f" Info: {info}")

# Un paso
accion = env.action_space.sample()
obs, reward, terminated, truncated, info = env.step(accion)

print(f"\n Después de step({accion}):")
print(f" Nueva observación: {obs}")
print(f" Recompensa: {reward}")
print(f" Terminado: {terminated}")
print(f" Truncado: {truncated}")

env.close()

# Significado de la observación en CartPole
print("\n" + "=" * 60)
print("SIGNIFICADO DE LA OBSERVACIÓN EN CARTPOLE")

```

```

print("=" * 60)
print("""
La observación es un vector de 4 valores:
  [0] Posición del carro      (-4.8 a 4.8)
  [1] Velocidad del carro    (-∞ a ∞)
  [2] Ángulo del poste       (-0.42 rad a 0.42 rad)
  [3] Velocidad angular      (-∞ a ∞)

Objetivo: Mantener el poste en equilibrio
Recompensa: +1 por cada paso que el poste sigue en pie
Termina cuando:
  - El poste se inclina más de 12°
  - El carro sale del límite
  - Se alcanzan 500 pasos (éxito)
""")

```

=====

EXPLORANDO GYMNASIUM

=====

Entorno: CartPole-v1

```

-----
Espacio de observación: Box([-4.8          -inf -0.41887903      -inf],
[4.8          inf 0.41887903          inf], (4,), float32)
  - Tipo: Box
  - Forma: (4,)
  - Límites: [-4.8          -inf -0.41887903      -inf] a [4.8
inf 0.41887903          inf]

```

Espacio de acciones: Discrete(2)

- Número de acciones: 2
- Acciones: 0 = empujar izquierda, 1 = empujar derecha

Después de reset():

Observación: [0.0273956 -0.00611216 0.03585979 0.0197368]
Info: {}

Después de step(0):

Nueva observación: [0.02727336 -0.20172954 0.03625453 0.32351476]
Recompensa: 1.0
Terminado: False
Truncado: False

=====

SIGNIFICADO DE LA OBSERVACIÓN EN CARTPOLE

=====

La observación es un vector de 4 valores:

[0]	Posición del carro	$(-4.8 \text{ a } 4.8)$
[1]	Velocidad del carro	$(-\infty \text{ a } \infty)$
[2]	Ángulo del poste	$(-0.42 \text{ rad a } 0.42 \text{ rad})$
[3]	Velocidad angular	$(-\infty \text{ a } \infty)$

Objetivo: Mantener el poste en equilibrio

Recompensa: +1 por cada paso que el poste sigue en pie

Termina cuando:

- El poste se inclina más de 12°
- El carro sale del límite
- Se alcanzan 500 pasos (éxito)

```
[52]: # EJEMPLO PRÁCTICO 12: Probando diferentes entornos

# Asegurarse de que gym está importado
try:
    gym
except NameError:
    import gymnasium as gym

# Entornos que siempre están disponibles (sin dependencias extra)
entornos_basicos = [
    ("CartPole-v1", "Equilibrar un poste sobre un carro"),
    ("MountainCar-v0", "Subir un coche a la cima de una montaña"),
    ("Acrobot-v1", "Balancear un brazo robótico hacia arriba"),
]

# Entornos que requieren dependencias adicionales
entornos_extra = [
    ("LunarLander-v3", "Aterrizar una nave espacial (requiere: pip install \
↳gymnasium[box2d])"),
]

print("=" * 70)
print("ENTORNOS POPULARES DE GYMNASIUM")
print("=" * 70)

for nombre, descripcion in entornos_basicos:
    try:
        env = gym.make(nombre)
        print(f"\n {nombre}")
        print(f" {descripcion}")
        print(f" Observaciones: {env.observation_space}")
        print(f" Acciones: {env.action_space}")

        # Probar un episodio aleatorio
```

```

obs, _ = env.reset()
recompensa_total = 0
pasos = 0

for _ in range(200):
    accion = env.action_space.sample()
    obs, recompensa, terminado, truncado, _ = env.step(accion)
    recompensa_total += recompensa
    pasos += 1
    if terminado or truncado:
        break

    print(f"    Prueba aleatoria: {pasos} pasos, recompensa =_{recompensa_total:.1f}")
    env.close()

except Exception as e:
    print(f"\n {nombre}: Error - {e}")

# Intentar entornos extra
print("\n" + "-" * 70)
print("ENTORNOS ADICIONALES (requieren instalación extra)")
print("-" * 70)

for nombre, descripcion in entornos_extra:
    try:
        env = gym.make(nombre)
        print(f"\n {nombre} - Disponible")
        print(f"    {descripcion}")
        print(f"    Observaciones: {env.observation_space}")
        print(f"    Acciones: {env.action_space}")
        env.close()
    except Exception as e:
        print(f"\n {nombre} - No instalado")
        print(f"    {descripcion}")

print("\n" + "=" * 70)
print(" Para instalar entornos adicionales:")
print("    pip install gymnasium[box2d]    # LunarLander, BipedalWalker")
print("    pip install gymnasium[atari]    # Juegos Atari")
print("=" * 70)

```

===== ENTORNOS POPULARES DE GYMNASIUM =====

CartPole-v1

Equilibrar un poste sobre un carro

```

    Observaciones: Box([-4.8          -inf -0.41887903          -inf], [4.8
inf 0.41887903          inf], (4,), float32)
    Acciones: Discrete(2)
    Prueba aleatoria: 11 pasos, recompensa = 11.0

```

MountainCar-v0

```

    Subir un coche a la cima de una montaña
    Observaciones: Box([-1.2 -0.07], [0.6 0.07], (2,), float32)
    Acciones: Discrete(3)
    Prueba aleatoria: 200 pasos, recompensa = -200.0

```

Acrobot-v1

```

    Balancear un brazo robótico hacia arriba
    Observaciones: Box([ -1.          -1.          -1.          -1.          -12.566371
-28.274334], [ 1.          1.          1.          1.          12.566371 28.274334],
(6,), float32)
    Acciones: Discrete(3)
    Prueba aleatoria: 200 pasos, recompensa = -200.0

```

ENTORNOS ADICIONALES (requieren instalación extra)

LunarLander-v3 - Disponible

```

    Aterrizar una nave espacial (requiere: pip install gymnasium[box2d])
    Observaciones: Box([ -2.5          -2.5          -10.          -10.
-6.2831855 -10.
-0.          -0.          ], [ 2.5          2.5          10.          10.
6.2831855 10.
1.          1.          ], (8,), float32)
    Acciones: Discrete(4)

```

=====

Para instalar entornos adicionales:

```

    pip install gymnasium[box2d]      # LunarLander, BipedalWalker
    pip install gymnasium[atari]      # Juegos Atari

```

=====

```

[53]: # =====
# PRÁCTICA FINAL: REINFORCEMENT LEARNING
# =====
# Nombre: [TU NOMBRE]
# Fecha: [FECHA]
# Entorno elegido: [NOMBRE DEL ENTORNO]
# =====

# Imports necesarios (gymnasium ya debería estar instalado)

```

```

import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
from collections import deque, defaultdict
import random

# Opcional: Para DQN (descomenta si lo usas)
# import torch
# import torch.nn as nn
# import torch.optim as optim

print(" Librerías importadas correctamente")

```

Librerías importadas correctamente

```

[54]: # PASO 1: Cargar y explorar el entorno
# =====

# TODO: Elige tu entorno
env = gym.make("CartPole-v1") # Cambia esto si quieres otro entorno

print("=" * 50)
print("ANÁLISIS DEL ENTORNO")
print("=" * 50)

# TODO: Analiza el espacio de estados y acciones
print(f"Espacio de observación: {env.observation_space}")
print(f"Espacio de acciones: {env.action_space}")

# TODO: Describe qué significa cada observación y acción

=====
ANÁLISIS DEL ENTORNO
=====
Espacio de observación: Box([-4.8          -inf -0.41887903      -inf],
[4.8          inf 0.41887903          inf], (4,), float32)
Espacio de acciones: Discrete(2)

```

```

[55]: # PASO 2: Implementar el agente
# =====

# TODO: Implementa tu agente aquí
# Puedes usar Q-Learning (con discretización), DQN, o tu propio algoritmo

class MiAgente:
    def __init__(self, env):
        # TODO: Inicializa tu agente
        pass

```

```

def seleccionar_accion(self, estado):
    # TODO: Implementa la selección de acción
    pass

def aprender(self, estado, accion, recompensa, siguiente_estado, terminado):
    # TODO: Implementa el algoritmo de aprendizaje
    pass

```

```

[56]: # PASO 3: Entrenar el agente
# =====

def entrenar(env, agente, n_episodios=500):
    """Función de entrenamiento"""
    recompensas = []

    for episodio in range(n_episodios):
        estado, _ = env.reset()
        recompensa_total = 0
        terminado = False
        truncado = False

        while not (terminado or truncado):
            # TODO: Completa el bucle de entrenamiento
            accion = agente.seleccionar_accion(estado)
            siguiente, recompensa, terminado, truncado, _ = env.step(accion)

            agente.aprender(estado, accion, recompensa, siguiente, terminado or
↪truncado)

            estado = siguiente
            recompensa_total += recompensa

        recompensas.append(recompensa_total)

        if episodio % 100 == 0:
            print(f"Episodio {episodio}: Recompensa = {recompensa_total:.1f}")

    return recompensas

# TODO: Entrena tu agente
# agente = MiAgente(env)
# recompensas = entrenar(env, agente)

```

```

[57]: # PASO 4: Visualizar resultados
# =====

```

```

# TODO: Grafica las curvas de aprendizaje

# plt.figure(figsize=(10, 5))
# plt.plot(recompensas)
# plt.xlabel('Episodio')
# plt.ylabel('Recompensa')
# plt.title('Curva de Aprendizaje')
# plt.show()

```

```

[58]: # PASO 5: Evaluar el agente entrenado
# =====

def evaluar(env, agente, n_episodios=10):
    """Evalúa el agente sin exploración"""
    recompensas = []

    for _ in range(n_episodios):
        estado, _ = env.reset()
        recompensa_total = 0
        terminado = False
        truncado = False

        while not (terminado or truncado):
            # TODO: Evalúa sin exploración (greedy)
            accion = agente.seleccionar_accion(estado) # Sin epsilon
            estado, recompensa, terminado, truncado, _ = env.step(accion)
            recompensa_total += recompensa

        recompensas.append(recompensa_total)

    print(f"Evaluación: {np.mean(recompensas):.1f} ± {np.std(recompensas):.1f}")
    return recompensas

# TODO: Evalúa tu agente
# evaluar(env, agente)

```